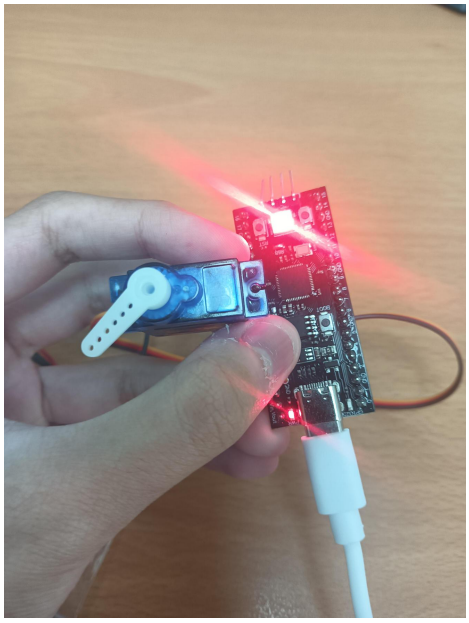
ws2812.c

```c
/**
 * Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
 *U13157004
 * SPDX-License-Identifier: BSD-3-Clause
 */

#include <stdio.h>
#include <stdlib.h>

#include "pico/stdlib.h"
#include "hardware/pio.h"
#include "hardware/clocks.h"
#include "ws2812.pio.h"

/**
 * NOTE:
 *  Take into consideration if your WS2812 is a RGB or RGBW variant.
 *
 *

 *  If it is RGBW, you need to set IS_RGBW to true and provide 4 bytes per
 *  pixel (Red, Green, Blue, White) and use urgbw_u32().
 *
 *  If it is RGB, set IS_RGBW to false and provide 3 bytes per pixel (Red,
 *  Green, Blue) and use urgb_u32().
 *
 *  When RGBW is used with urgb_u32(), the White channel will be ignored (off).
 *
 */
#define IS_RGBW false
#define NUM_PIXELS 150

#ifdef PICO_DEFAULT_WS2812_PIN
#define WS2812_PIN PICO_DEFAULT_WS2812_PIN
#else
// default to pin 2 if the board doesn't have a default WS2812 pin defined
#define WS2812_PIN 23
#endif

// Check the pin is compatible with the platform
#if WS2812_PIN >= NUM_BANK0_GPIOS
#error Attempting to use a pin>=32 on a platform that does not support it
#endif

static inline void put_pixel(PIO pio, uint sm, uint32_t pixel_grb) {
    pio_sm_put_blocking(pio, sm, pixel_grb << 8u);
}

static inline uint32_t urgb_u32(uint8_t r, uint8_t g, uint8_t b) {
    return
            ((uint32_t) (r) << 8) |
```

```
52              ((uint32_t) (g) << 16) |
53              (uint32_t) (b);
54  }
55
56  static inline uint32_t urgbw_u32(uint8_t r, uint8_t g, uint8_t b, uint8_t w) {
57      return
58              ((uint32_t) (r) << 8) |
59              ((uint32_t) (g) << 16) |
60              ((uint32_t) (w) << 24) |
61              (uint32_t) (b);
62  }
63
64  void pattern_snakes(PIO pio, uint sm, uint len, uint t) {
65      for (uint i = 0; i < len; ++i) {
66          uint x = (i + (t >> 1)) % 64;
67          if (x < 10)
68              put_pixel(pio, sm, urgb_u32(0xff, 0, 0));
69          else if (x >= 15 && x < 25)
70              put_pixel(pio, sm, urgb_u32(0, 0xff, 0));
71          else if (x >= 30 && x < 40)
72              put_pixel(pio, sm, urgb_u32(0, 0, 0xff));
73          else
74              put_pixel(pio, sm, 0);
75      }
76  }
77
78  void pattern_random(PIO pio, uint sm, uint len, uint t) {
79      if (t % 8)
80          return;
81      for (uint i = 0; i < len; ++i)
82          put_pixel(pio, sm, rand());
83  }
84
85  void pattern_sparkle(PIO pio, uint sm, uint len, uint t) {
86      if (t % 8)
87          return;
88      for (uint i = 0; i < len; ++i)
89          put_pixel(pio, sm, rand() % 16 ? 0 : 0xffffffff);
90  }
91
92  void pattern_greys(PIO pio, uint sm, uint len, uint t) {
93      uint max = 100; // let's not draw too much current!
94      t %= max;
95      for (uint i = 0; i < len; ++i) {
96          put_pixel(pio, sm, t * 0x10101);
97          if (++t >= max) t = 0;
98      }
99  }
100
101 typedef void (*pattern)(PIO pio, uint sm, uint len, uint t);
102 const struct {
103
104     pattern pat;
105     const char *name;
```



```
} pattern_table[] = {
        {pattern_snakes,  "Snakes!"},
        {pattern_random,  "Random data"},
        {pattern_sparkle, "Sparkles"},
        {pattern_greys,   "Greys"},
};

int main() {
    //set_sys_clock_48();
    stdio_init_all();
    printf("WS2812 Smoke Test, using pin %d\n", WS2812_PIN);

    // todo get free sm
    PIO pio;
    uint sm;
    uint offset;

    // This will find a free pio and state machine for our program and load it for us
    // We use pio_claim_free_sm_and_add_program_for_gpio_range (for_gpio_range variant)
    // so we will get a PIO instance suitable for addressing gpios >= 32 if needed and supported by the hardware
    bool success = pio_claim_free_sm_and_add_program_for_gpio_range(&ws2812_program, &pio, &sm, &offset, WS2812_PIN, 1, true);
    hard_assert(success);

    ws2812_program_init(pio, sm, offset, WS2812_PIN, 800000, IS_RGBW);

    int t = 0;
    while (1) {
        // int pat = rand() % count_of(pattern_table);
        // int dir = (rand() >> 30) & 1 ? 1 : -1;
        // puts(pattern_table[pat].name);
        // puts(dir == 1 ? "(forward)" : "(backward)");
        // for (int i = 0; i < 1000; ++i) {
        //     pattern_table[3].pat(pio, sm, NUM_PIXELS, t);
        //     //pattern_table[pat].pat(pio, sm, NUM_PIXELS, t);
        //     sleep_ms(10);
        //     t += dir;
        // }

        put_pixel(pio, sm, urgb_u32(0xff, 0, 0));
        sleep_ms(1000);
        put_pixel(pio, sm, urgb_u32(0, 0xff, 0));
        sleep_ms(1000);
        put_pixel(pio, sm, urgb_u32(0, 0, 0xff));
        sleep_ms(1000);
    }

    // This will free resources and unload our program
    pio_remove_program_and_unclaim_sm(&ws2812_program, pio, sm, offset);
}

```